AEKD – GESTION DE TONTINE

Rapport d'analyse globale et d'état d'avancement

MVP web – Amicale des Enfants Kamkop de Douala

3 octobre 2026 · branche master (commit e041b9a + modifications locales)

Élément	Valeur
Objet	Analyse complète du projet : ce qui est fait, ce qui est en cours, ce qui reste à faire
Références	CLAUDE.md, docs/regles-metier.md, docs/decisions.md, docs/rapport-analyse.md, code source, historique Git (19 commits)
Méthode	Lecture du code backend et frontend, inventaire des endpoints et des autorisations, exécution de la suite de tests backend, vérification du typage TypeScript, comparaison avec le cahier des charges
Verdict	**MVP fonctionnellement complet à environ 85 %.** Le flux de démonstration de bout en bout est couvert ; restent quelques écarts au cahier des charges, des décisions métier à faire valider, les tests end-to-end et la stabilisation du déploiement.

≈ 85 %	74 / 74	13 / 13	16
Avancement global MVP	Tests backend réussis	Fonctions MVP présentes	Décisions à valider

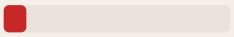
1. Synthèse exécutive

En neuf jours de développement (24 septembre – 2 octobre 2026), le projet est passé d'un dépôt vide à une application web complète : un backend Spring Boot (Java 21, PostgreSQL, Flyway, JWT) d'environ 7 350 lignes et 12 migrations, et un frontend Angular / Angular Material d'environ 11 300 lignes (TypeScript + gabarits) organisé en 34 composants fonctionnels.

- **Fait :** l'ensemble des 13 blocs fonctionnels du périmètre MVP (CLAUDE.md §3) existe côté serveur *et* côté interface : authentification, utilisateurs, membres, séances, cotisations (assistant multi-étapes), participants/bénéficiaires, paiements et justificatifs, validation, sanctions, fonds et épargne, prêts, remboursements, tableaux de bord, audit.
- **En cours (non commité) :** refonte de la page Membres avec une synthèse par membre (nouvel endpoint `GET /api/dashboard/members`) et correction de `application.yml`.
- **Reste à faire :** corriger 6 écarts au cahier des charges, faire valider 16 décisions métier (intérêts et sanctions en priorité), écrire les tests d'API/sécurité et end-to-end, fiabiliser le déploiement et préparer la démonstration.
- **Point d'alerte immédiat :** la version commitée de `application.yml` (commit « Déploiement Backend ») contient une URL de base de données au format Railway (`${{...}}`) non interprétable par Spring et un port par défaut à 5432 : le backend tel qu'il est versionné ne démarre pas en local. La correction existe dans la copie de travail mais n'est pas commitée.

2. Avancement par phase (CLAUDE.md §35)

Les pourcentages sont une estimation qualitative fondée sur l'inventaire du code et des tests ; ils ne mesurent pas un temps restant.

Phase	Contenu	Avancement	%	Commentaire
Phase 1	Analyse, architecture, modèle de données		100 %	Modèle complet (17 concepts + SessionBeneficiary), règles métier et décisions documentées.
Phase 2	Backend Spring Boot		92 %	Tous les modules livrés. Manquent : modification/suppression de cotisation et de séance, droits trésorier sur les politiques de prêt.
Phase 3	Tests backend		70 %	74 tests d'intégration des services (base PostgreSQL isolée) tous verts. Aucun test de contrôleur / sécurité HTTP (MockMvc).
Phase 4	Frontend Angular		90 %	Tous les écrans du MVP. Manque un écran « Fonds » dédié (module cash) ; page Membres en refonte.
Phase 5	Intégration front / back		85 %	Intercepteurs JWT et erreurs, gardes de rôle, appels à toutes les API. Typage TypeScript sans erreur.
Phase 6	Tests end-to-end		10 %	Aucun test E2E automatisé ; 6 fichiers de tests unitaires frontend (utilitaires).
Phase 7	Préparation de la démonstration		50 %	Jeu de données de démonstration en place ; déploiement Railway amorcé mais configuration non stabilisée.

3. Chronologie du développement

Date	Réalisations (commits)
24/09/2026	Initialisation du dépôt et du backend ; User/Role/Member et login ; séances, cotisations, périodes, déclaration de paiement ; justificatifs, fonds, sanctions, prêts/remboursements, audit ; tableaux de bord ; données de démonstration.
25/09/2026	Swagger, base de test isolée, gestion utilisateurs/membres, auto-inscription ; initialisation Angular (auth + tableau de bord) ; charte AEKD (orange/marron/gris) ; écrans Séances, Cotisations (assistant), déclaration / validation des paiements.
26/09/2026	Paiements partiels réels et vue des statuts par participant ; écrans prêts et remboursements ; écrans sanctions ; préparation des déploiements frontend et backend (Railway).
27/09 – 02/10	Travaux non commités : page Membres enrichie, synthèse par membre côté backend, rapport d'analyse (docs/rapport-analyse.md).

4. Ce qui a été fait – Backend

Architecture en couches Controller → Service → Repository, DTO en entrée/sortie (aucune entité JPA exposée), sécurité par `@PreAuthorize` et contrôles d'appartenance dans les services, montants en NUMERIC/BigDecimal, documentation OpenAPI/Swagger.

Module	Contenu livré	État
auth / security	Login JWT, auto-inscription (compte en attente de validation), filtre JWT, configuration Spring Security	**Fait**
user / member	CRUD utilisateurs (ADMIN), validation des inscriptions, statut PENDING_VALIDATION/ACTIVE/DISABLED, consultation et modification des membres	**Fait**
session	Séance à date unique (lieu, membre récepteur, bénéficiaires de séance) – création, liste, détail	**Partiel**
contribution	Définitions (FIXED/VOLUNTARY, obligatoire, PUBLIC/PRIVATE, destination, DRAFT/ACTIVE/INACTIVE), activation, participants, bénéficiaires, périodes, statut par participant (PAID, PENDING, PARTIAL, LATE, NOT_PAID)	**Partiel**
paiements	PaymentService → ManualPaymentService ; déclaration (participant uniquement), paiements partiels, validation/rejet par le trésorier, interdiction de l'auto-validation	**Fait**
storage	FileStorageService → LocalFileStorageService ; justificatifs JPG/JPEG/PNG/PDF (10 Mo max), consultation	**Fait**
fund	Fonds commun (TONTINE_FUND) et épargne individuelle (INDIVIDUAL_SAVINGS) calculés à partir des seuls paiements validés, strictement séparés	**Fait**
sanction	SanctionRule (MONETARY / IN_KIND, seuil configurable) distincte d'AppliedSanction ; détection des candidats en retard, application manuelle, annulation	**Fait**
loan	LoanPolicy configurable (min/max, durée, taux, épargne minimale, ancienneté, prêts actifs), demande avec évaluation détaillée des règles (LoanRuleEvaluation), approbation/rejet	**Fait**
repayment	Remboursements multiples, refus au-delà du solde, solde restant, passage APPROVED → IN_PROGRESS → REPAID	**Fait**
dashboard	Tableau de bord ADMIN/TRESORIER et MEMBRE ; synthèse par membre (en cours, non commité)	**Fait**
audit	AuditLog alimenté par 10 services (séances, cotisations, paiements, sanctions, prêts, remboursements, utilisateurs) ; consultation ADMIN	**Fait**
config	DemoDataSeeder : admin, trésorier, 5 membres fictifs, séance, cotisations connues, paiements, règle de sanction, politique de prêt, demandes de prêt	**Fait**

Migrations Flyway

Version	Contenu
V1 – V3	Rôles, utilisateurs, membres, rôles initiaux
V4	Séances
V5 – V7	Cotisations, participants, bénéficiaires, périodes, transactions, justificatifs
V8 / V9	Sanctions (règles et appliquées) / politiques de prêt, prêts, évaluations, remboursements
V10 – V12	Journal d'audit ; statut utilisateur ; séance à date unique avec lieu, hôte et bénéficiaires

5. Ce qui a été fait – Frontend

Angular 22 en composants autonomes chargés à la demande, Angular Material, formulaires réactifs, intercepteurs (jeton JWT, gestion d'erreurs), gardes `authGuard` et `roleGuard`, service worker (application installable), charte graphique AEKD.

Écran / route	Rôles	Fonctions
/login, /register	Public	Connexion, auto-inscription
/dashboard	Tous	Tableau de bord différencié ADMIN/TRESORIER et MEMBRE (cartes statistiques)
/sessions, /sessions/:id	Tous	Liste, création (dialogue), détail affichant chaque cotisation séparément
/contributions, /new, /:id	Tous / staff	Liste, assistant de création multi-étapes, détail, participants, bénéficiaires, statuts par participant
/my-contributions	Membre	Cotisations du membre, déclaration de paiement, envoi de justificatif
/payments	ADMIN, TRESORIER	Paiements en attente, visualisation du justificatif, validation / rejet
/sanctions	Tous	Règles, candidats en retard, application/annulation (staff) ; mes sanctions (membre)
/loan-policies	Tous (édition ADMIN)	Politiques de prêt, activation / désactivation
/my-loans, /loans	Membre / staff	Demande avec check-list des règles, décision, remboursements, solde
/members	ADMIN, TRESORIER	Liste des membres, édition (refonte en cours avec cartes de synthèse)
/users, /audit	ADMIN	Comptes et validation des inscriptions ; journal d'audit (export CSV)

Composants partagés : badge de statut, dialogue de confirmation, dialogue d'information, menu d'actions, barre de recherche, filtres, pagination, états de page (chargement / vide / erreur), export CSV (sortie de périmètre validée, décision n° 29).

6. Conformité au périmètre MVP et au flux de démonstration

Étape du flux CLAUDE.md §33	Couverture
1-2. Le trésorier se connecte et crée la séance « Octobre 2026 »	**OK**
3. Il crée la cotisation 50 000 FCFA, choisit les participants, ajoute une règle de sanction, active	**OK**
4-5. Un membre voit la cotisation et déclare 50 000 FCFA via MTN MoMo avec référence, date et justificatif	**OK**
6. Le trésorier consulte le justificatif et valide le paiement	**OK**
7. Le fonds commun est mis à jour ; l'opération est tracée dans l'audit	**OK**
8. Le membre demande un prêt de 300 000 FCFA ; la politique est vérifiée	**OK**
9. Le trésorier approuve le prêt	**OK**
10. Remboursement de 50 000 FCFA ; solde restant recalculé	**OK**

Le flux est couvert fonctionnellement par le code et par les tests de services ; il n'a pas encore été rejoué et enregistré de bout en bout de manière automatisée (voir Phase 6).

7. Qualité et tests (vérifiés le 3 octobre 2026)

Vérification	Résultat
Suite de tests backend (`mvn test`, copie de travail)	**74 tests, 0 échec, 0 erreur** – 12 classes
Typage frontend (`tsc --noEmit`)	**Aucune erreur**
Build de production Angular	Non relancé (provoque une saturation mémoire sur le poste) ; dernier build : 02/10/2026
Tests unitaires frontend	6 fichiers (utilitaires, statut de période, service des séances) – non exécutés ici
Tests HTTP / sécurité (MockMvc)	Aucun
Tests end-to-end	Aucun

Classe de test	Tests	Classe de test	Tests
UserServiceTest	10	ContributionTransactionFlowTest	11
SessionServiceTest	9	ContributionDefinitionServiceTest	7
DashboardServiceTest	6	PaymentProofServiceTest	6
LoanFlowTest	6	SanctionFlowTest	6
MemberServiceTest	4	AuthServiceTest	3
FundServiceTest	3	AuditLogServiceTest	3

8. Travaux en cours non commités

- **Page Membres enrichie** (≈ 2 400 lignes modifiées) : cartes statistiques par membre, dialogue d'édition repensé, nouveau composant `member-stat-card` et `info-dialog`.
- **Backend** : `GET /api/dashboard/members` (MembersOverviewResponse, MemberSummaryResponse) avec situation des cotisations obligatoires, épargne, encours de prêts et sanctions par membre ; tests ajoutés à DashboardServiceTest (verts).
- **application.yml** : retour à une configuration par variables d'environnement avec valeurs locales par défaut (port 8080) – corrige la version commitée non fonctionnelle.
- **Fichiers parasites** : 5 journaux de plantage JVM (`hs_err_pid*.log`, `replay_pid*.log`) à la racine et dans backend/, non ignorés par .gitignore.

Recommandation : vérifier visuellement la page Membres, puis commiter ces travaux en deux commits distincts (configuration / fonctionnalité) et ajouter `hs_err_pid*.log` *et* `replay_pid*.log` *au .gitignore.*

9. Écarts par rapport au cahier des charges

Ces points diffèrent de CLAUDE.md : chacun doit être corrigé ou validé comme choix assumé.

#	Écart constaté	Référence	Action proposée
1	Pas de modification ni de suppression de cotisation (`PUT/DELETE /api/contributions/{id}`) ; seule l'activation existe	§28	Implémenter (au moins sur les cotisations en brouillon)
2	Pas de modification d'une séance (seulement création / consultation)	§6, §26	Ajouter la modification, tracée dans l'audit
3	Les politiques de prêt ne sont configurables que par l'ADMIN	§20	Ouvrir au TRESORIER ou valider le choix
4	L'enregistrement d'un remboursement est réservé au TRESORIER ; le membre ne peut que consulter	§6	Décider : déclaration par le membre puis validation, ou choix assumé
5	Pas d'écran dédié aux fonds (module *cash*) ; fonds et épargne visibles seulement via les tableaux de bord	§29	Créer l'écran Fonds (fonds commun + épargne par membre)
6	Statut PARTIAL et séance à date unique non reflétés dans CLAUDE.md / regles-metier.md	§7, §13	Mettre la documentation à jour

10. Ce qui reste à faire (priorisé)

Priorité	Tâche	Taille	Dépend de
P1	Commiter la correction de application.yml et stabiliser la configuration de déploiement (profil local / profil Railway)	S	–
P1	Finaliser, vérifier et commiter la page Membres et l'endpoint de synthèse	S	–
P1	Faire valider les décisions d'intérêts (taux, calcul, capital + intérêt) et des sanctions	–	Bureau AEKD
P1	Rejouer le scénario §33 complet sur l'application lancée et corriger les anomalies	M	–
P2	Modification / suppression de cotisation et modification de séance (écarts 1 et 2)	M	–
P2	Écran Fonds dédié (écart 5)	M	–
P2	Trancher les droits trésorier / membre (écarts 3 et 4) et adapter API + écrans	S	Décision
P2	Tests d'API et de sécurité (MockMvc) : vérification des rôles sur chaque endpoint	M	–
P2	Tests end-to-end du flux de démonstration (ex. Playwright)	M	–
P3	Intégrer le calcul des intérêts dans le solde des prêts	M	Décision n° 14 / 19
P3	Contrôle des fonds disponibles à l'approbation d'un prêt	M	Décision n° 18
P3	Notifications push (demandées, décision n° 28) – le service worker est déjà en place	L	Clarification
P3	Préparation production : stockage cloud des justificatifs, secret JWT, suppression des comptes de démo, sauvegardes PostgreSQL	M	Choix d'hébergement
P3	Mise à jour de CLAUDE.md, regles-metier.md et d'un README de lancement	S	–

Taille : S = quelques heures, M = une à deux journées, L = plusieurs jours (ordre de grandeur indicatif).

11. Décisions métier à valider (docs/decisions.md)

Conformément au principe de non-invention, aucune de ces valeurs n'est codée en dur : elles sont configurables ou en attente. 16 décisions sont au statut « À VALIDER » ; 12 sont « DÉCIDÉ » ; 1 est « REPORTÉ » (application mobile).

N°	Sujet	Traitement actuel dans l'application	Impact
3	Montant de « Voir bébé » et destination	Montant configurable	Faible
4	Montant / règles de « Banque / Épargne »	Configurable, destination épargne individuelle	Faible
5	Usage précis des fonds de « Ration »	Rattachée au fonds commun	Faible
6	Délais de paiement	Échéance saisie pour chaque période	Moyen
7	Règles de retard (seuil)	Seuil configurable dans la règle de sanction	Moyen
8	Montants des sanctions financières	Saisis librement dans la règle	Moyen
9	Sanctions en nature	Description libre	Faible
10	Application automatique ou proposée	Candidats détectés automatiquement, application manuelle	Moyen
11-13	Prêts : montant min, max, durée maximale	Paramètres de la LoanPolicy	Moyen
14	Taux d'intérêt et mode de calcul	Taux stocké ; solde calculé sur le capital seul	**Élevé**
15-17	Épargne minimale, ancienneté, nombre de prêts actifs	Paramètres optionnels de la LoanPolicy	Moyen
19	Remboursement : capital, + intérêt, + frais	Capital seul en attendant	**Élevé**
18	Mécanique des fonds disponibles pour les prêts (principe décidé)	Non contrôlée à l'approbation	**Élevé**
28	Notifications push (statut noté « VALIDER », à clarifier)	Non implémentées	Moyen

12. Risques et points d'attention

Risque	Conséquence	Mesure proposée
Configuration commitée non fonctionnelle	Un clone du dépôt ne démarre pas en local ; déploiement fragile	Commiter la correction ; profils Spring séparés (local / prod)
Intérêts non calculés	Solde des prêts sous-estimé en usage réel	Valider les décisions 14 et 19 avant toute production
Fonds disponibles non contrôlés	Approbation possible d'un prêt supérieur aux fonds	Définir la mécanique (décision 18)
Stockage local des justificatifs	Perte des fichiers sur un hébergement à conteneurs éphémères (Railway)	Volume persistant ou stockage cloud via FileStorageService
Pas de tests HTTP / E2E	Régressions de permissions non détectées	MockMvc + un scénario E2E
Plantages JVM / mémoire locale	Builds et tests interrompus (journaux hs_err présents)	Limiter la mémoire (-Xmx), ne pas lancer build front et tests back en parallèle
Identifiants de démonstration et secret JWT par défaut	Faille de sécurité en production	Désactiver le seeder et imposer APP_JWT_SECRET en production

13. Conclusion et prochaines étapes

Le MVP AEKD-Tontine est une application réellement fonctionnelle et non une maquette : toutes les briques du périmètre sont implémentées, testées au niveau des services et reliées à l'interface. Les séparations exigées par le cahier des charges (définition / période / déclaration / validation, fonds commun / épargne, règle de sanction / sanction appliquée, politique de prêt configurable) sont respectées.

Pour atteindre un MVP démontrable sans réserve, l'ordre recommandé est :

- **Cette semaine :** commiter la configuration et la page Membres, nettoyer les journaux JVM, rejouer le scénario §33.
- **Ensuite :** combler les écarts 1, 2 et 5 (modification des cotisations et séances, écran Fonds) et ajouter les tests HTTP et E2E.
- **En parallèle :** présenter au bureau de l'AEKD la liste des décisions de la section 11, en priorité intérêts, remboursement et sanctions.
- **Avant production :** stockage persistant des justificatifs, secrets, suppression des données de démonstration, sauvegardes.